



Синтез и анализ на алгоритми за факултети ФКСТ, ФТК, ФЕТТ

проф. д-р Даниела Минковска
каб. 2324, кат. ПКТ, ФКСТ, ТУ-София
daniela@tu-sofia.bg



Лекция 5

Структури от данни. Дървета - видове, представяне, обхождане, алгоритми за запис и четене.

“АЛГОРИТМИ + СТРУКТУРИ ОТ ДАННИ = ПРОГРАМИ”

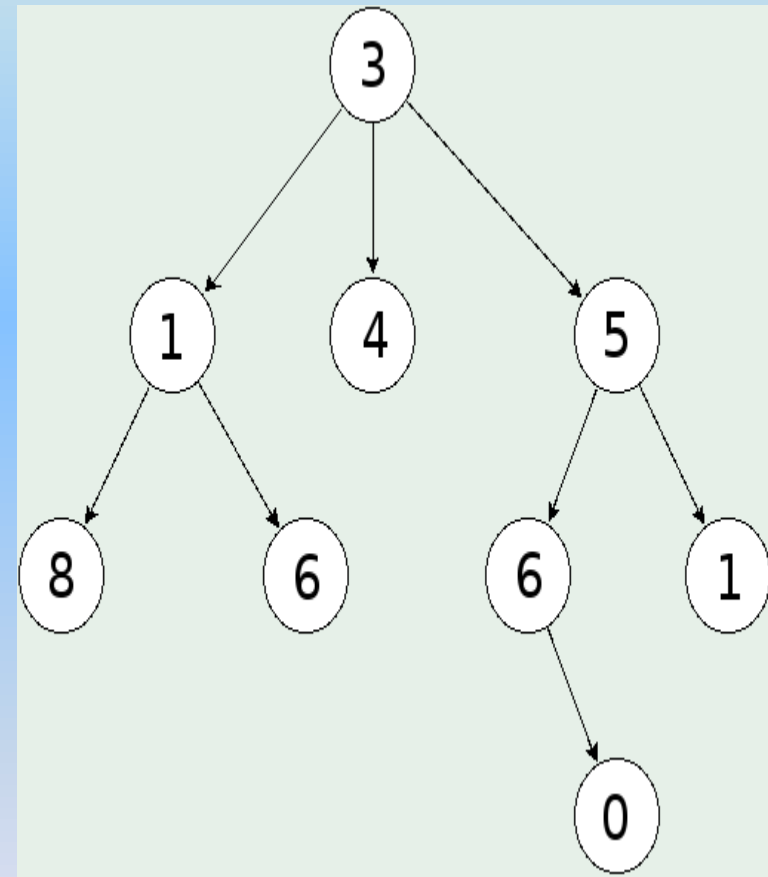
Н. УИРТ

Дървета. Определение. Видове

Дефинция: дървото е свързан граф, без цикли но с някои допълнителни свойства (рекурсивна структура).

Видове:

- **Неориентирано** дърво – ненасочен свързан ацикличен граф, при който за корен е избран един възел;
- **Ориентирано** дърво – насочен ацикличен граф, в който има *един възел без входяща дъга*, наречен **корен**, а всички останали възли имат само по една входяща и произволен брой изходящи дъги.



*От корена до всеки възел има един
единствен път!*



Дървета. Основни понятия

Разглеждаме само ориентирани дървета.

Възел, който **няма изходящи дъги** се нар. краен (**листо**).

Посоката на дъгите е **отгоре-надолу** (стрелки не се чертаят).

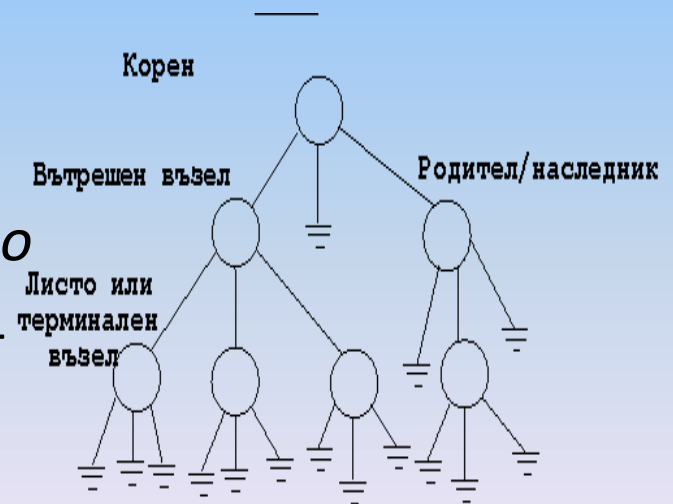
Нека между възли **x** и **y** има **път** $\longrightarrow$ **x** е **предшественик**, а **y** е **приемник (наследник)**.

Когато **(x,y)** е **дъга** $\longrightarrow$ **x** е **възел-родител (баща)**, **y**-**възел-дете (син)**.

*Всеки възел е приемник(предшественик)
на себе си. Същински приемници на даден
възел **x** са тези, които са различни от него*

Всички приемници на даден възел и свързващите ги дъги образуват **поддърво**.

Листо се нарича **връх без наследници**!





Дървета. Основни понятия

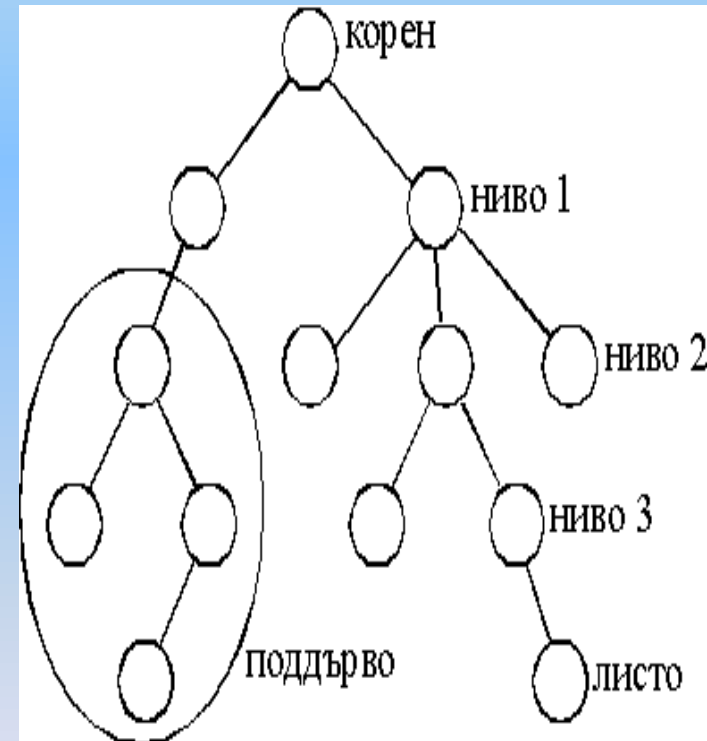
Разстояние между два възела, между които има път, е **броят на клоните** на този път.

Дълбочина на възел – разстоянието между корена и възела.

Височина на възел – най-дългото разстояние от възела до краен възел.

Височината на корена е и височина на дървото.

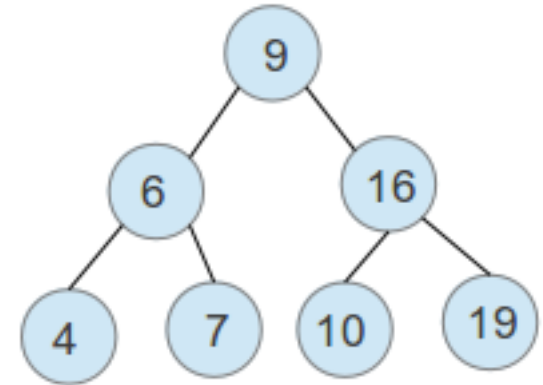
Ниво на възел - разликата между височината на дървото и дълбочината на възела.



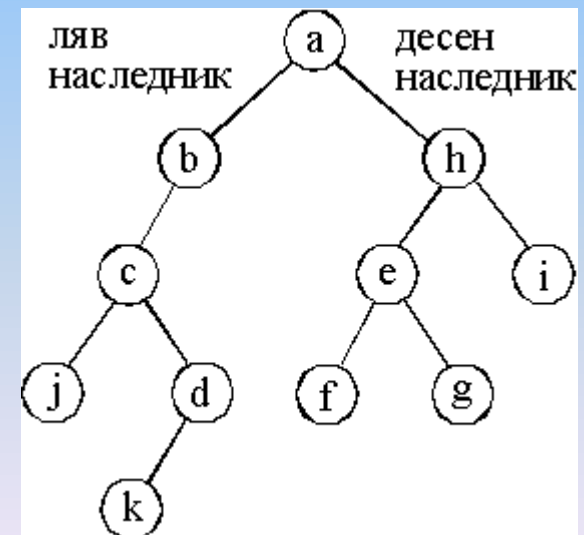
При двоично дърво всеки възел има най-много два приемника!

Дървета. Двоични дървета

Дефиниция: Дърво, в което броят на наследниците на върховете е **0, 1 или 2** се нарича **двоично дърво**. Всеки връх има **ляв** и **десен наследник** (може и празно множество).



Рекурсивно определение на двоично дърво: Крайно множество от елементи (възли), което е **или празно**, или се състои от **корен (възел)**, свързан с **две непресичащи се двоични дървета** (поддървета) - ляво и дясно поддърво.

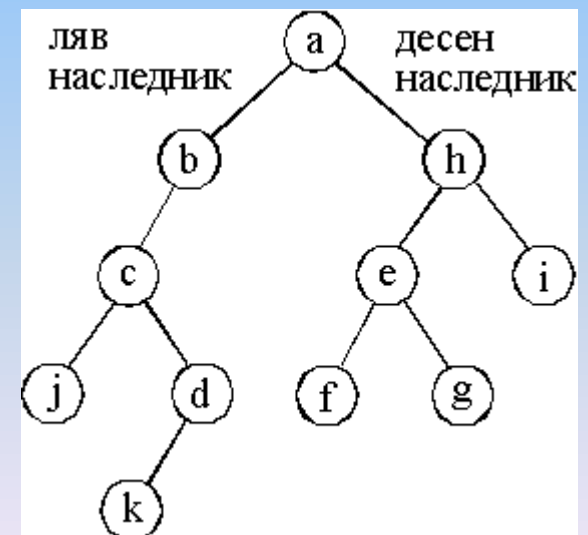
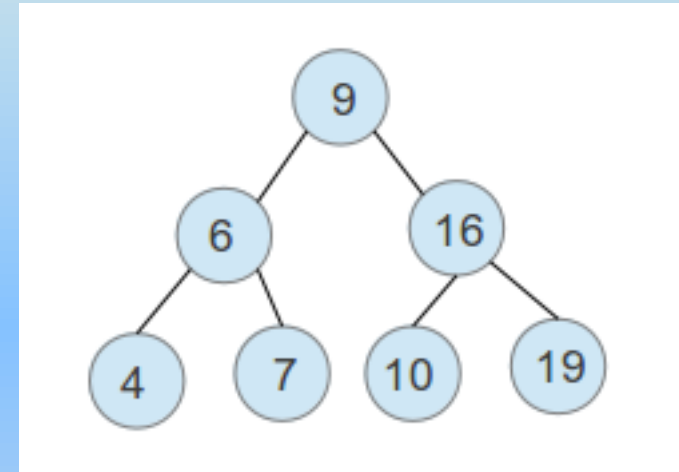




Дървета. Двоични дървета

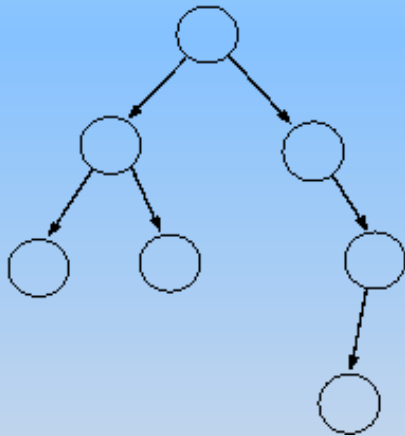
Двоичните дървета имат следните характеристики:

- Всеки възел има **до две деца**.
- Стойността на **лявото дете** е **по-малка** от стойността на **родителя**.
- Стойността на **дясното дете** е **по-голяма** от стойността на **родителя**.

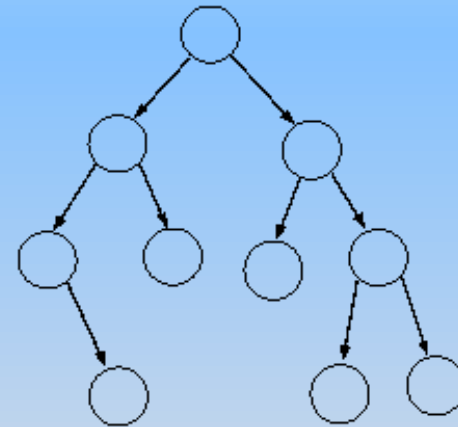


Дървета. Балансирани дървета

Дефиниция: дърво, за което и изпълнено условието: **разликата** във височините на поддърветата на всеки негов възел е най-много **единица** се нарича **балансирано** дърво



НЕбалансирано дърво



Балансирано дърво



Дървета. Задаване на дърво

Задаването на дърво може да се представи по 5 начина:

- Статично – с масив баща;
- Статично – с два масива;
- Динамично;
- Статично задаване на двоично дърво;
- Динамични задаване на двоично дърво.

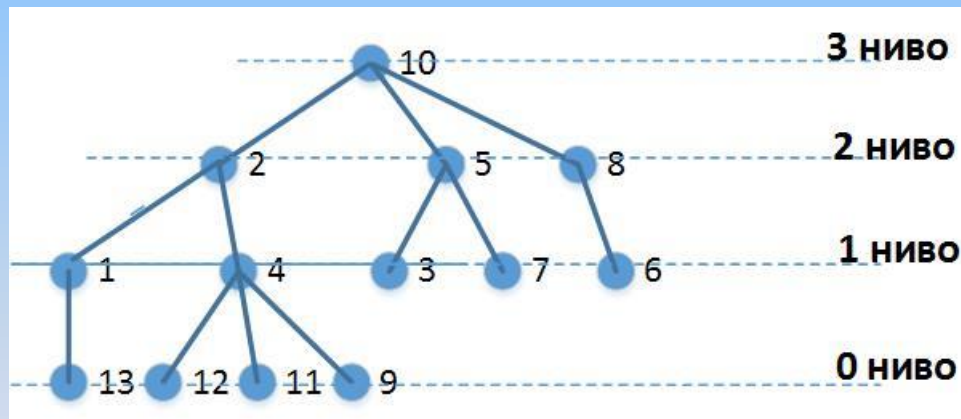
Ако означим с n – броят на възлите, а с k – броят на клоните на дървото:



Дървета. Задаване на дърво

1. **Статично** – с **масив баща** с дължина n : - за всеки възел се задава възела - баща.

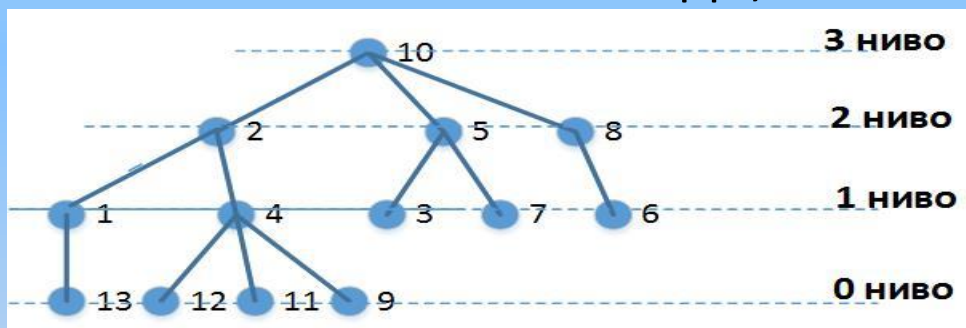
Предимство: икономичен начин за паметта. Лесно движение от даден възел към предшествениците му. **Недостатък:** трудно се извършва обратно движение – от един възел към приемниците му.



V - възел	1	2	3	4	5	6	7	8	9	10	11	12	13
Баща [V]	2	10	5	2	10	8	5	10	4	0	4	4	1

Дървета. Задаване на дърво

2. Статично – с два масива (*съсед* и *начало*) – в масива *съсед* последователно за всеки възел се записват възел–баща и възлите–синове отляво надясно, а в масив *начало* за всеки възел *x* се записва индекса на клетка от масива *съсед*, в която е записан възела-баща на *x*.



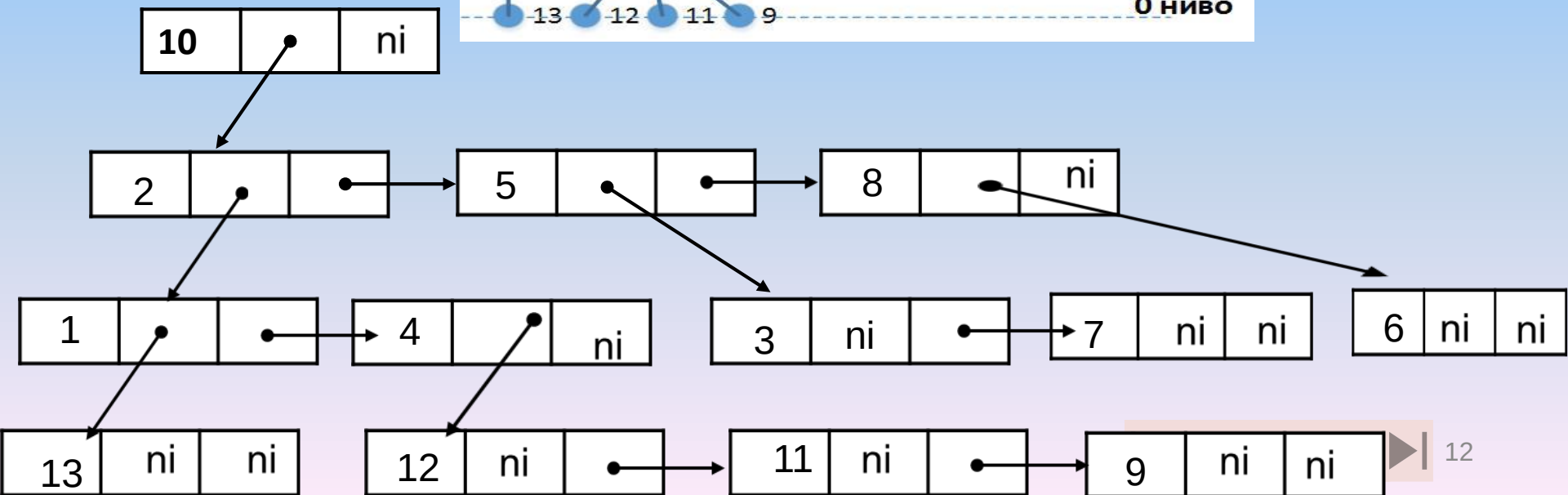
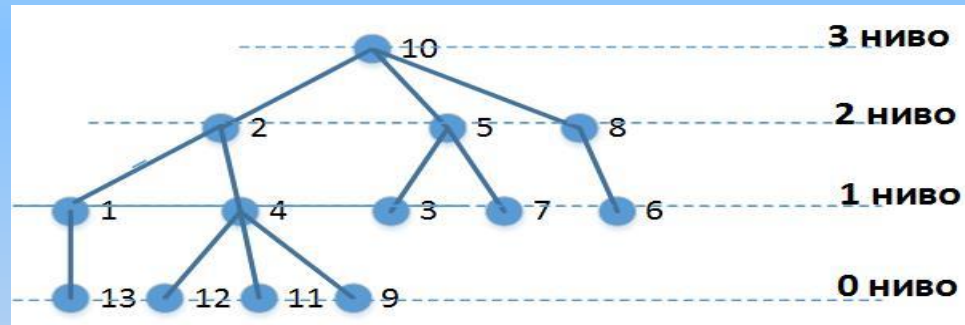
	①		②			③		④				
i	1	2	3	4	5	6	7	8	9	10	...	
съсед [i]	2	13	10	1	4	5	2	12	11	9	...	

V	1	2	3	4		
Начало[V]	1	3	6	7		



Дървета. Задаване на дърво

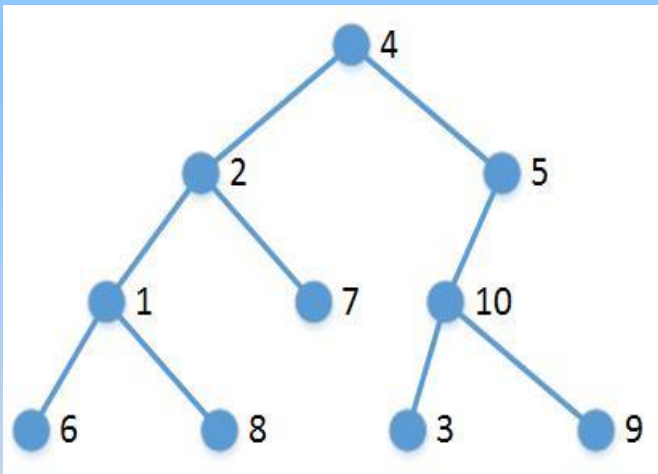
3. Динамично – за всеки възел **x** има **елемент с 3 полета**- първото за **номер на възела** второто за **указател към списък на синовете** му и третото – **следващият син на бащата на x**. (Може да има и четвърто поле – **указател към елемента на възела-баща на x**)





Дървета. Задаване на дърво

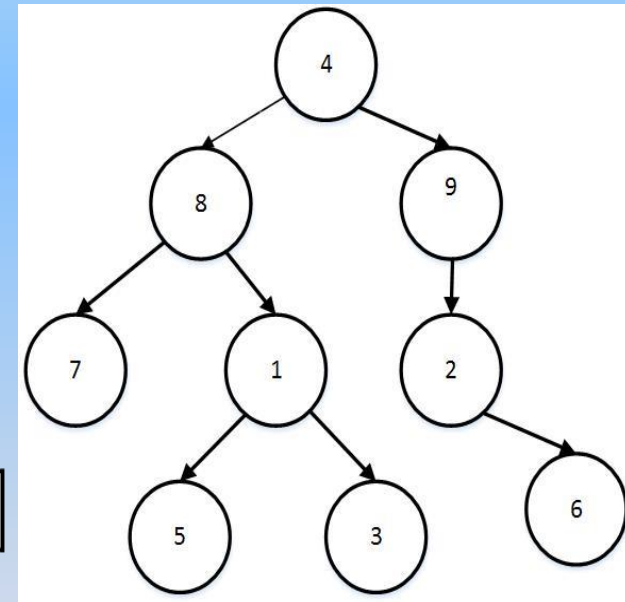
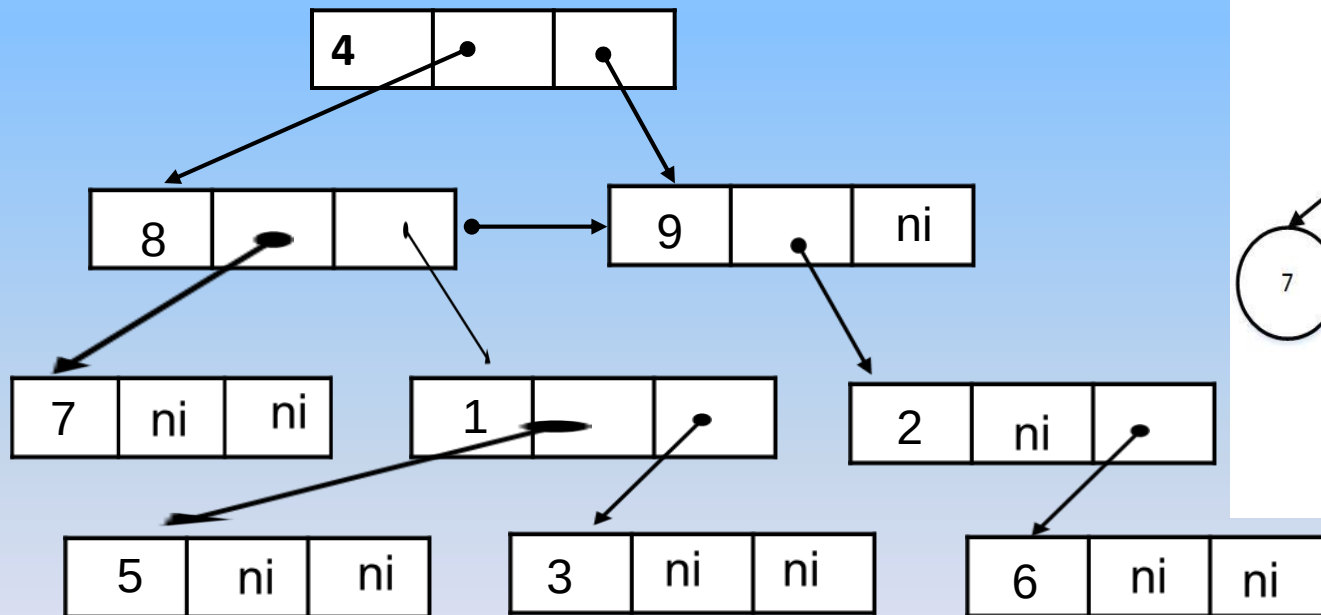
4. Двоично дърво – може да се представи **статично** с масив от записи с две полета, едното от които е ляв син, а другото – десен син.



	ляв	десен
1	6	8
2	1	7
3	0	0
4	2	5
5	10	0
6	0	0
7	0	0
8	0	0
9	0	0
10	3	9

Дървета. Задаване на дърво

5. Двоично дърво – може да се представи **динамично** с елементи от три полета, едното за номер на възел, второто – указателно за елемента ляв син и третото – указателно за елемента десен син.





Дървета. Основни оперции

Основни операции със структурата дърво:

- . Инициализиране на дърво
- . Обхождане на дърво
- . Добавяне на елемент в дърво
- . Премахване на елемент от дърво
- . Търсене на елемент.



Дървета. Инициализация на дърво

Инициализиране на структурата дърво:

На **указателя към корена** на дървото се присвоява стойност **NULL** при дефинирането на структурата:

```
struct elem  
    {char key; elem *left, *right;} *root=NULL;
```




Дървета. Обхождане на дърво

Обхождането означава **посещаване** на **всеки възел по веднъж**. Т.к. дървото е нелинейна структура, обхождането му може да става по различни начини.

Начини за обхождане на дърво – прав, обратен и вътрешен ред.
Прав и обратен – за всяко дърво; вътрешен – само за двоично.

- в прав ред – **PREORDER**
- във вътрешен ред (или симетрично обхождане) – **INORDER**
- в обратен ред – **POSTORDER**

Обхождането винаги започва от корена!



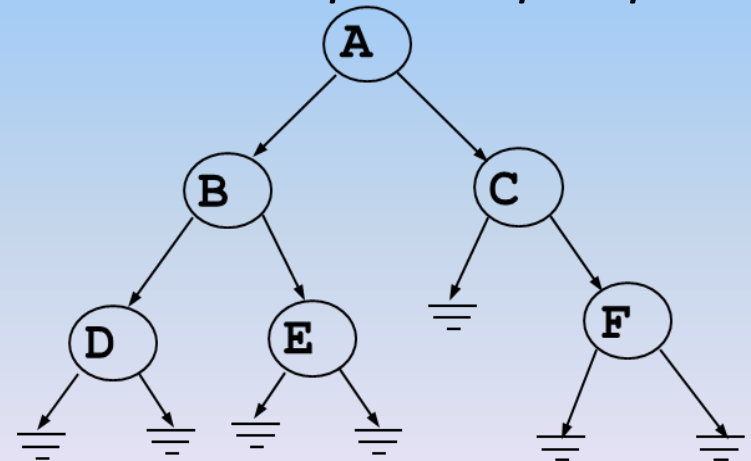
Дървета. Обхождане на дърво

1. Обхождане в прав ред – PREORDER:

При обхождането в прав ред възлите се посещават в следната последователност: **корен - ляво поддърво - дясно поддърво**, което за долния пример на двоично дърво ще даде такава последователност: **ABDECF**.

Функция за рекурсивно обхождане на двоично дърво в прав ред:

```
void preorder(elem *t)
{
    if (t)
    {
        cout<<t->key<<" ";
        preorder(t->left);
        preorder(t->right);
    }
}
```



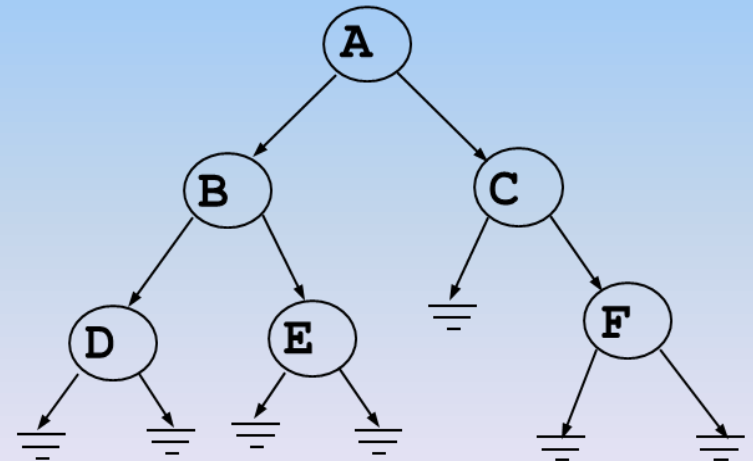
Дървета. Обхождане на дърво

2. Обхождане във вътрешен ред (или симетрично обхождане) – INORDER:

Обхождането във вътрешен ред се извършва така: **ляво поддърво** - **корен** - **дясно поддърво**, т.е. за долния пример - **DBEACF**.

Функция за рекурсивно обхождане на двоично дърво във вътрешен ред:

```
void inorder(elem *t)
{
    if (t)
    {
        inorder(t->left);
        cout<<t->key<<" ";
        inorder(t->right);
    }
}
```



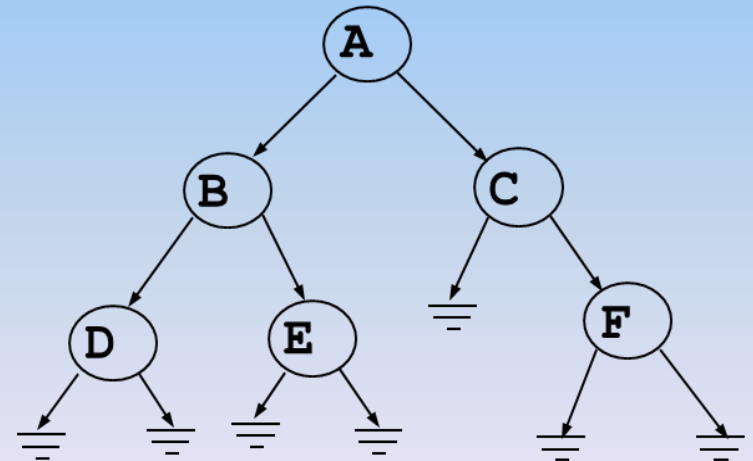
Дървета. Обхождане на дърво

3. Обхождане в обратен ред – POSTORDER:

Обхождането в обратен ред означава следната последователност на посещаваните възли: **ляво поддърво - дясно поддърво - корен**, за примера тя е **DEBFCA**:

Функция за рекурсивно обхождане на двоично дърво в обратен ред:

```
void postorder(elem *t)
{
    if (t)
    {
        postorder(t->left);
        postorder(t->right);
        cout<<t->key<<" ";
    }
}
```





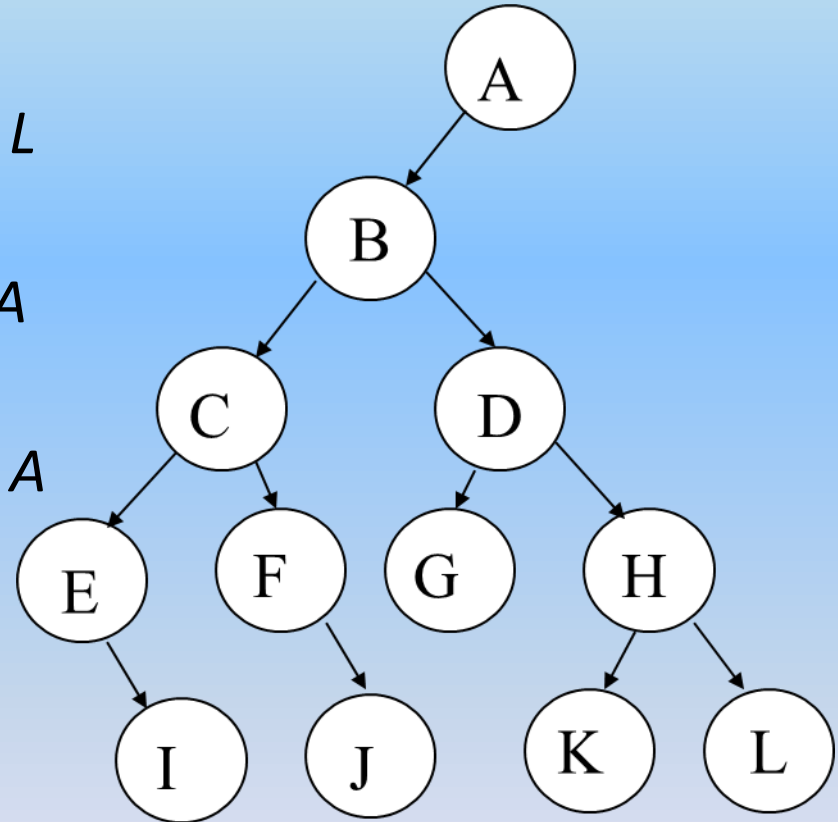
Дървета. Обхождане на дърво

Пример: за обхождане на двоично дърво:

Preorder: (КЛД): *A B C E I F J D G H K L*

Inorder: (ЛКД): *E I C F J B G D K H L A*

Postorder: (ЛДК): *I E J F C G K L H D B A*

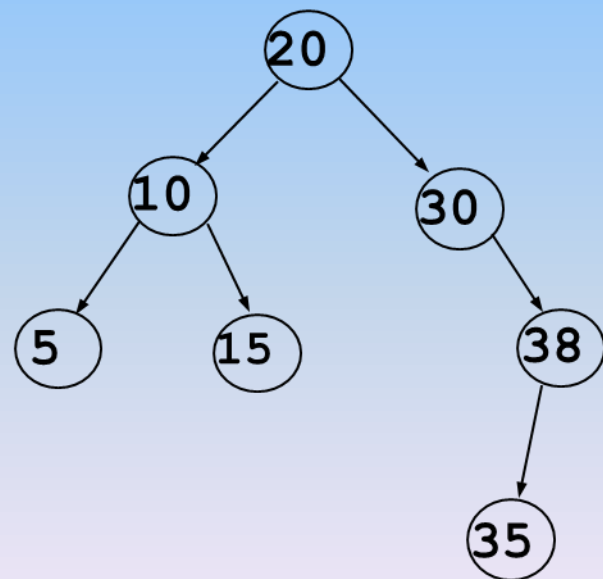


Дървета. Добавяне на елемент в дърво

Операциите - добавяне, изтриване и търсене на елемент - ще разгледаме върху една често използвана разновидност на структурата - **двоични дървета за търсене** или **подредени двоични дървета** (или **BST** - Binary Search Tree).

Едно двоично дърво се нарича **двоично дърво за търсене**, ако за всеки връх с **ключова стойност K** е вярно:

- всички елементи в лявото му поддърво имат ключови стойности по-малки от **K**;
- всички елементи в дясното му поддърво имат ключови стойности по-големи от **K**;
- ключовите стойности са уникални.



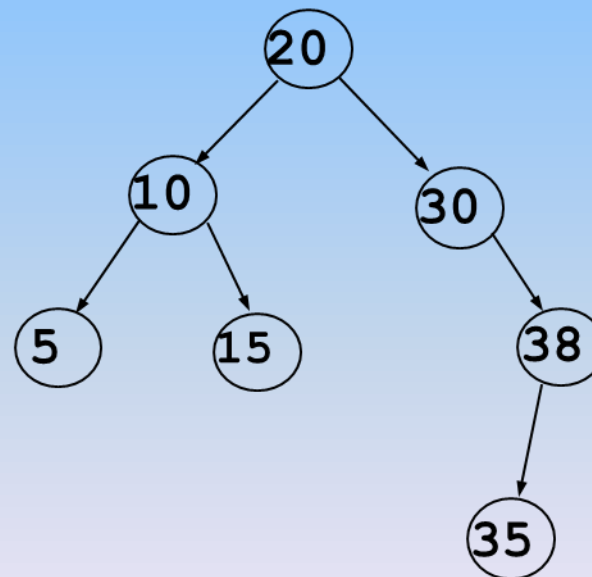


Дървета. Добавяне на елемент в дърво

Пример: Симетричното обхождане на двоично дърво за търсене извежда ключовите стойности на структурата, подредени в нарастващ ред.

Така за примера функция INORDER ще даде следния резултат:

5 10 15 20 30 35 38





Дървета. Добавяне на елемент в дърво

Нови елементи се **добавят** в структурата във вид на **листа**.
Разгледаната долу операция за добавяне на нов възел в двоично дърво за търсене е реализирана като **рекурсивна функция**.

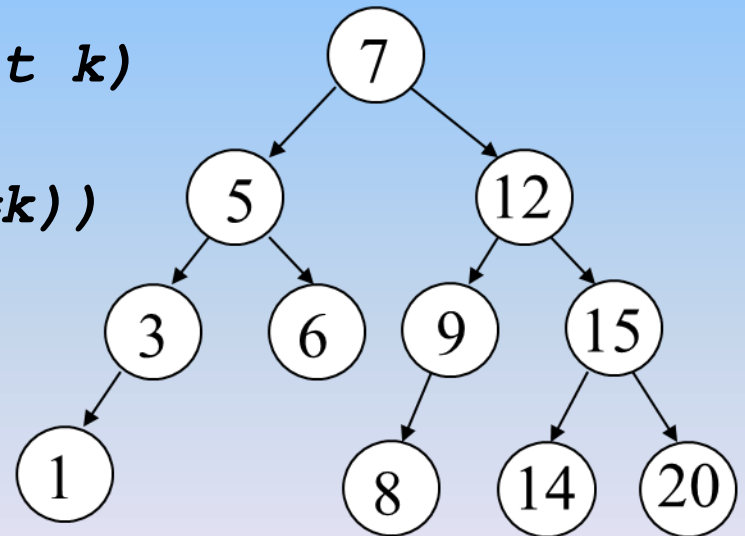
```
void add(int n, elem* &t)
{
    if (t==NULL)
    {
        t=new elem;
        t->key=n;
        t->left=t->right=NULL;
    }
    else
    {
        if (t->key<n)
            add(n, t->right);
        else
            add(n, t->left);
    }
}
```


Дървета. Търсене на елемент в дърво

Двете примерни функции връщат като резултат указател към търсения елемент. Ако елементът не принадлежи на дървото, върнатата стойност е NULL

Итеративно търсене на елемент в подредено двоично дърво:

```
elem *search_iter(elem *t, int k)
{
    while ((t!=NULL) && (t->key!=k))
        if (t->key<k)
            t=t->right;
        else
            t=t->left;
    return t;
}
```

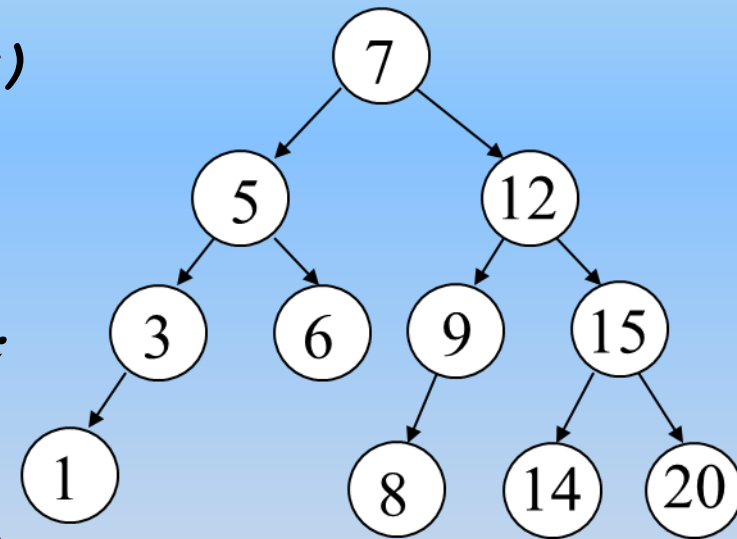




Дървета. Търсене на елемент в дърво

Рекурсивно търсене на елемент в подредено двоично дърво:

```
elem *search_rec(elem *t, int k)
{
    if (t!=NULL)
        if (t->key<k)
            t=search_rec(t->right, k);
        else
            if (t->key>k)
                t=search_rec(t->left, k);
    return t;
}
```





Дървета. Търсене на елемент в дърво

Още един вариант на итеративна функция за търсене в двоично дърво:

```
elem * &search(int k) // функцията връща адреса на указател  
{  
    // към търсения елемент  
    elem **p=&root;  
    for (;;)   
    {  
        if (*p==NULL) return *p;  
        if (k<(*p)->key) p=&(*p)->left;  
        else  
        if (k>(*p)->key) p=&(*p)->right;  
        else  
            return *p;  
    }  
}
```



Дървета. Изтриване на елемент от дърво

Операцията **премахване** на възел от двоично дърво за търсене не трябва да нарушава подредбата на структурата.

Възможни са следните варианти:

- Ако премахваният елемент е **листо**, неговото отстраняване **не нарушава структурата**;
- Ако премахваният елемент има **само един наследник**, той **замества изтривания възел**;
- Ако премахваният елемент има **два наследника**, то изтриваният възел **се замества или с най-десния** (най-големия) елемент **от лявото му поддърво** или с **най-левия** (най-малкия) елемент от **дясното му поддърво**.



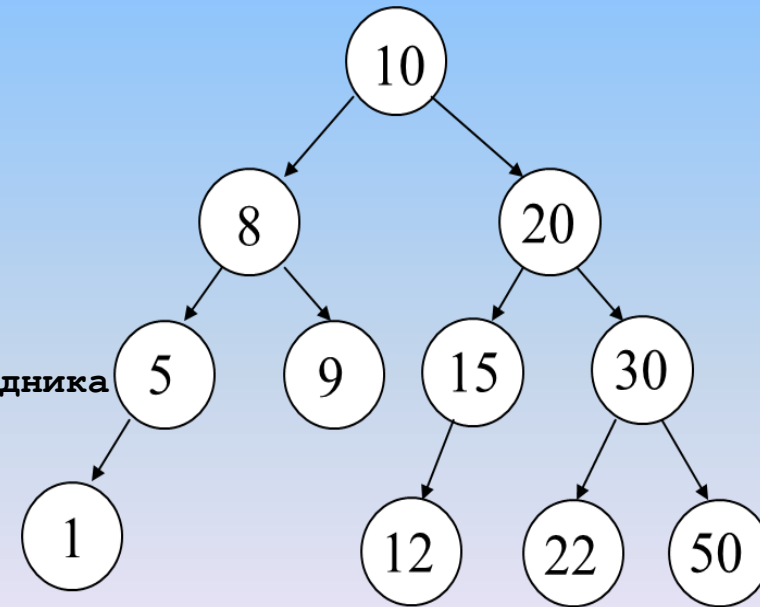
Дървета. Изтриване на елемент от дърво

Изтриване на възел:

```
struct tree {
    int key;
    tree *left,*right;
};

tree *root=NULL,*z;

...
int del(int k)
{
    tree * &p=search(k), *p0=p, **qq, *q;
    // search(k) - функция, която търси в дървото възел
    // с ключова стойност k и връща адреса на указател към него
    if (p==NULL) return 0; //Възелът не е намерен
    if ((p->right==NULL))
    {
        p=p->left; delete p0;
    }
    else
    if (p->left==NULL)
    {
        p=p->right; delete p0;
    }
    else //Възелът е с два наследника
    {
        qq=&p->left;
        while ((*qq)->right)
        qq=&(*qq)->right;
        p->key=(*qq)->key;
        q=*qq;
        *qq=q->left;
        delete q;
    }
    return 1; }
```





Дървета. Пример

Пример: създаване на двоично дърво, пресмятане на неговата височина и брой възли с последващо изобразяване на дървото. (Елементите на дървото са от символен тип.)

```
//TREE'S PARAMETERS
```

```
#include <iostream.h>
```

```
struct elem
```

```
{ char key; elem *left, *right;} *root=NULL;
```

```
void add(char n, elem* &t); //prototypes
```

```
int count(elem *t);
```

```
int height(elem *t);
```

```
void printnode(char n, int h);
```

```
void show (elem *t, int h);
```

```
void main()
```

```
{
```

```
char c;
```

```
cout<<"Enter some characters to be placed in a binary tree (followed by /): \n";
```

```
while ((cin>>c)&&(c!='/'))
```

```
add(c, root);
```

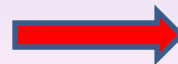
```
cout<<"\nNode's count: "<<count(root);
```

```
int h=height(root)+1; //+1 - for NULL-nodes
```

```
cout<<"\nTree's height: "<<h<<' \n';
```

```
show(root, h);
```

```
}
```





Дървета. Пример

```
void add(char n, elem* &t)
{
    if (t==NULL)
    {
        t=new elem;
        t->key=n;
        t->left=t->right=NULL;
    }
    else
    {
        if (t->key<n)
            add(n,t->right);
        else
            add(n,t->left);
    }
}

int count (elem *t)
{
    if (t==NULL) return 0;
    return count(t->left)+count(t->right)+1;
}
```





Дървета. Пример

```
int height (elem *t)
{
    if (t==NULL) return -1;
    int u=height(t->left), v=height(t->right);
    if (u>v) return u+1;
    else
        return v+1;
}

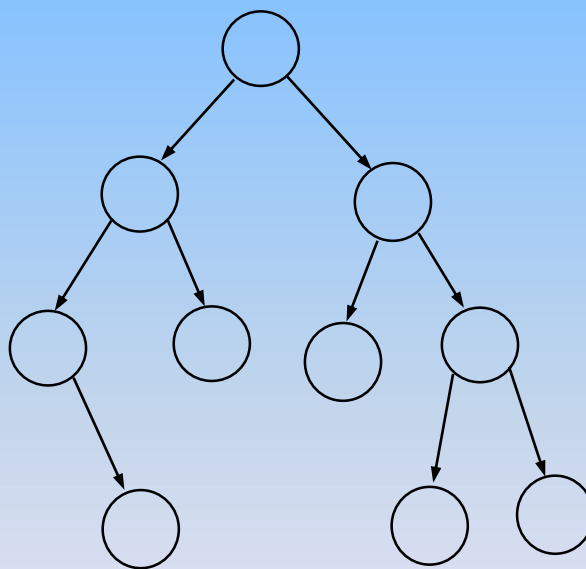
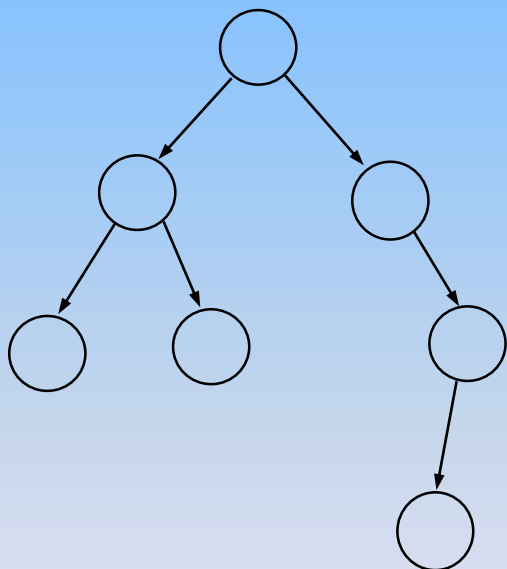
void printnode (char n, int h)
{
    for (int i=0; i<h; i++)
        cout<<" ";
    cout<<n<<"\n";
}

void show (elem *t, int h)
{
    if (t==NULL)
    {
        printnode('*', h);
        return;
    }
    show(t->right, h+1);
    printnode(t->key, h);
    show(t->left, h+1);
}
```




Дървета. Балансирани дървета

Всяко двоично дърво може да бъде **балансирано**. **Двоично дърво** се нарича **балансирано** или **балансирано по височина**, ако разликата във височините на поддърветата на всеки негов възел е най-много 1:

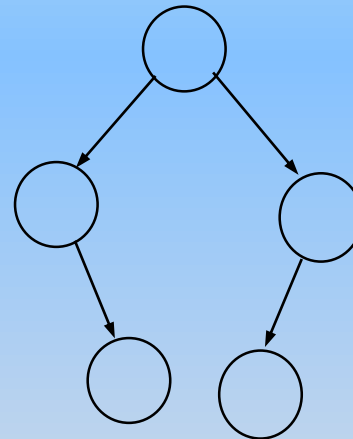
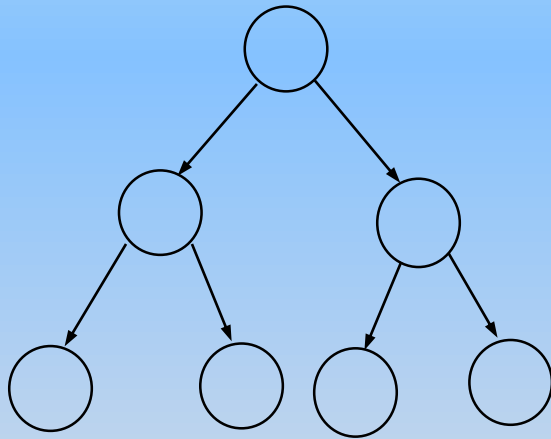


Лявото дърво не е балансирано. Дясното е балансирано



Дървета. Балансирани дървета

Едно двоично дърво се нарича **идеално балансирано** или с **балансирано тегло**, ако **всеки негов възел има ляво и дясно поддърво, в които броят на възлите се различава най-много с 1**:



Очевидно е, че всяко идеално балансирано дърво (с балансирано тегло) също така е дърво с балансирана височина. *Обратното твърдение не е вярно.*



Дървета. Балансирани дървета

Идеално балансираното дърво може да бъде **пълно**, ако за всеки възел разликата в броя на възлите в неговите поддървета е **нула**. Лявото дърво на горната фигура е пълно.

За да бъде двоичното дървото **пълно**, **броят на възлите** му трябва да е **нечетен**. Ако **K** е броя на нивата в дървото (коренът се брои за 0-во ниво), то броят на вътрешните възлите в пълното двоично дърво **N** е равен на **$2^{K+1} - 1$** .

Балансът обикновено е важен фактор при работа с двоични дървета за търсене, където е от значение времето за търсене и не трябва да има отделни дълги клони.

Балансираните дървета често се наричат AVL-дървета в чест на Adelson-Velski и Landis - двамата учени, създали най-много алгоритми за този клас дървовидни структури.



Дървета. Балансирани дървета

Основните операции с балансираните дървета - добавяне, премахване и търсене на елемент с зададена ключова стойност - имат сложност $O(\log_2 N)$.

Ако $h(N)$ е височината на балансирано двоично дърво с N вътрешни възела, то:

$$\log_2(N+1) \leq h(N) \leq 1.4404 \log_2(N+2) - 0.328$$



Дървета. Балансирани дървета

Всяко двоично дърво може да бъде балансирано.

Има **4 основни операции** за балансиране на двоично дърво, наречени **ротации**:

- **Дясна ротация;**
- **Лява ротация;**
- **Ляво – дясна ротация;**
- **Дясно – лява ротация.**



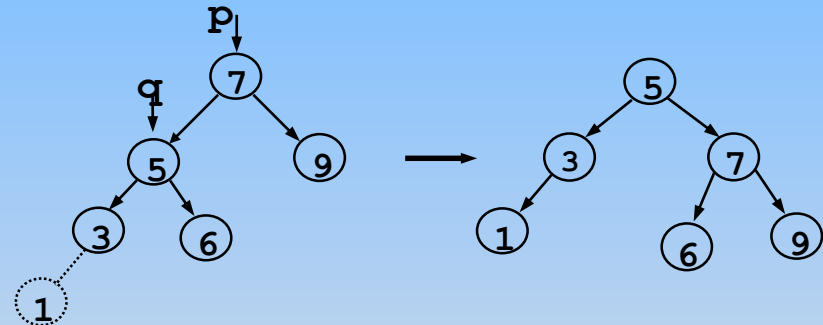
Дървета. Балансирани дървета

1. **Дясна ротация** - Тази операция е необходима тогава, когато във вече балансираното двоично дърво се **добавя** елемент **към най-левия клон**:

Функция за **дясна ротация** в двоично дърво за търсене:

```
void R_rot(tree * &p)
```

```
{  
    tree *q;  
    if(p->bal==-1) {  
        q=p->left;  
        if(q->bal==-1) {  
            p->left=q->right;  
            q->right=p;  
            p->bal=0;  
            p=q;  
        }  
    }  
}
```





Дървета. Балансирани дървета

2. Лява ротация - Тази операция е необходима тогава, когато във вече балансираното двоично дърво се **добавя** елемент **към най-десния клон**:

Функция за **лява ротация** в двоично дърво за търсене:

```
void L_rot(tree * &p)
```

```
{
```

```
    tree *q;
```

```
    if(p->bal==1) {
```

```
        q=p->right;
```

```
        if(q->bal==1) {
```

```
            p->right=q->left;
```

```
            q->left=p;
```

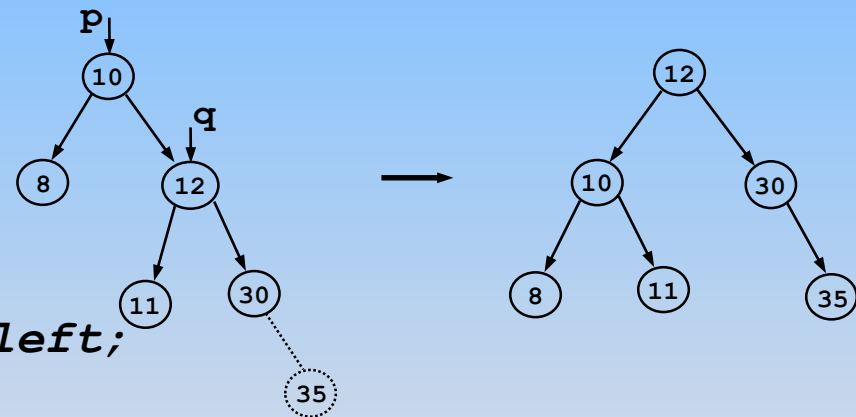
```
            p->bal=0;
```

```
            p=q;
```

```
        }
```

```
    }
```

```
}
```



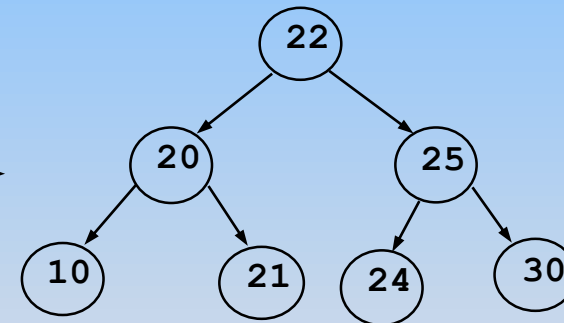
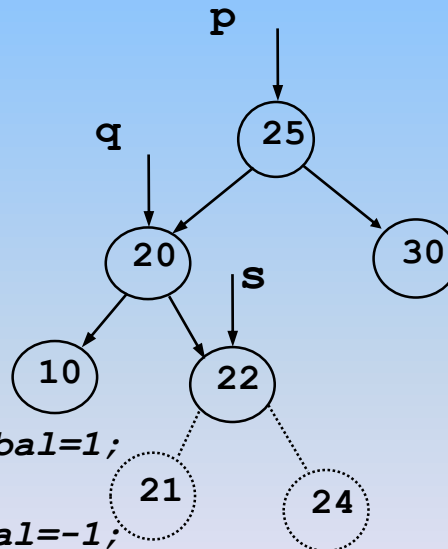


Дървета. Балансирани дървета

3. Ляво-дясна ротация - Тази операция е необходима тогава, когато във вече балансираното двоично дърво се **добавя елемент към дясното листо на по-високото ляво поддърво**, в случая 21 или 24:

Функция за ляво-дясна ротация в двоично дърво за търсене:

```
void L_R_rot(tree * &p)
{
    tree *q,*s;
    if(p->bal== -1)
    {
        q=p->left;
        if(q->bal==1)
        {
            s=q->right;
            q->right=s->left;
            s->left=q;
            p->left=s->right;
            s->right=p;
            if(s->bal== -1) p->bal=1;
            else p->bal=0;
            if(s->bal==1) q->bal= -1;
            else q->bal=0;
            p=s;
        }
    }
}
```



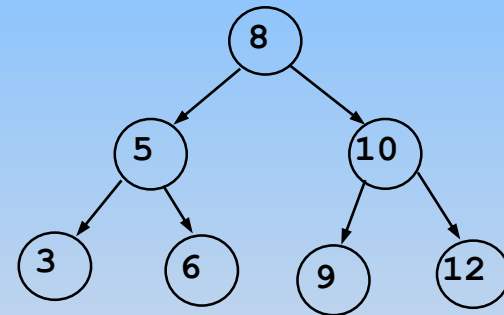
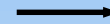
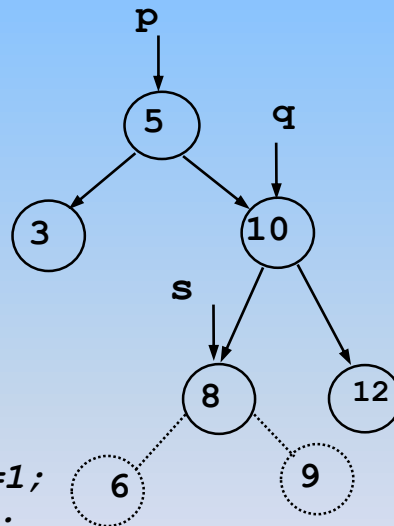


Дървета. Балансирани дървета

4. Дясно - лява ротация - Тази операция е необходима тогава, когато във вече балансираното двоично дърво се **добавя елемент към лявото листо на по-високото дясно поддърво**, в случая 6 или 9:

Функция за **дясно-лява ротация** в двоично дърво за търсене:

```
void R_L_rot(tree * &p)
{
    tree *q,*s;
    if(p->bal==1)
    {
        q=p->right;
        if(q->bal==-1)
        {
            s=q->left;
            q->left=s->right;
            s->right=q;
            p->right=s->left;
            s->left=p;
            if(s->bal==1) p->bal=1;
            else p->bal=0;
            if(s->bal==-1) q->bal=1;
            else q->bal=0;
        }
        p=s;
    }
}
```

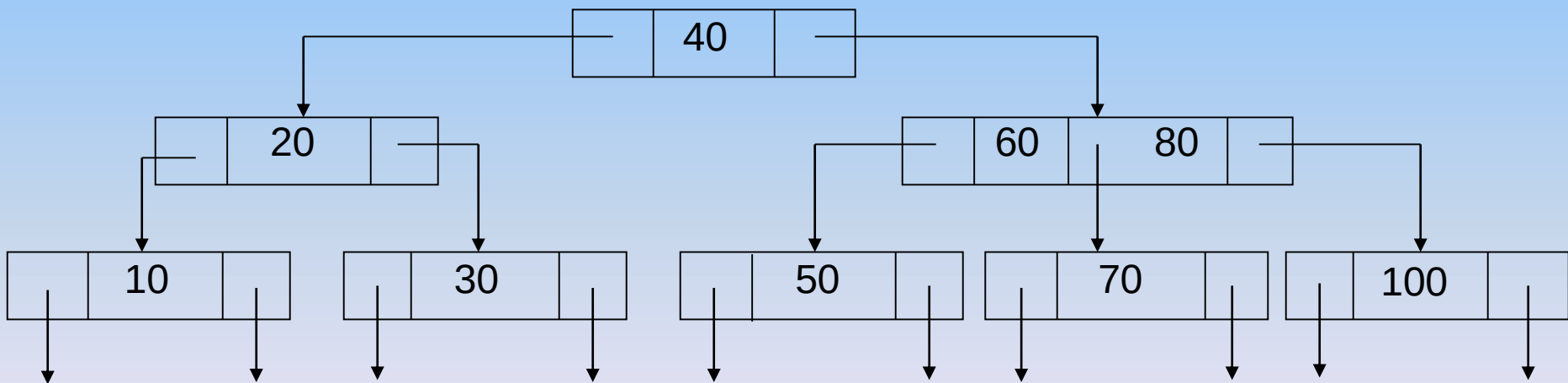




Дървета. 2-3 дървета

Това са идеално балансирано дърво за претърсване, в което:

- всеки вътрешен възел съдържа една ключова стойност и два указателя или две ключови стойности и три указателя (наследника);
- всяко листо съдържа една или две ключови стойности;
- всички пътища от корена до листата имат еднакви дължини.

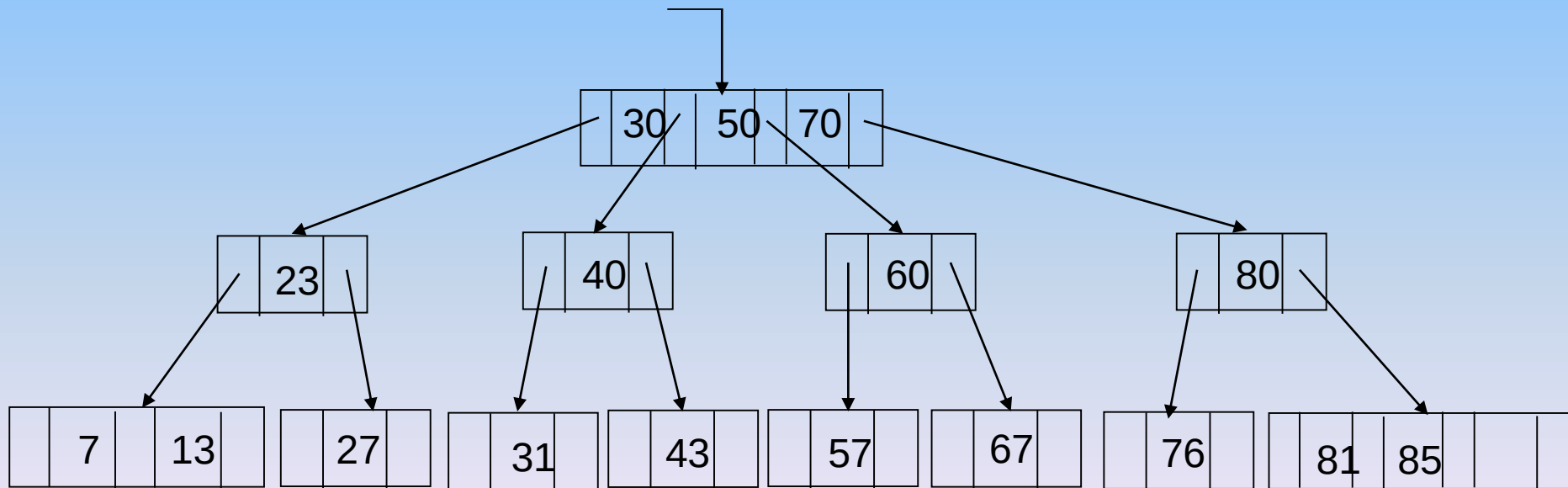




Дървета. 2-3-4 дървета

2-3-4 - дърво е дърво T със следните свойства:

- T е идеално балансирано дърво;
- всеки вътрешен връх има 2, 3 или 4 наследника;
- всички пътища от корена до произволно листо имат еднаква дължина:





Дървета. 2-3-4 дървета

Съществуват ефективни алгоритми за работа с 2-3-4 -дървета, които гарантират сложност на основните операции (добавяне, търсене, изтриване) от порядъка на:

$O(\log_2 n)$.

Съществуват начини за преобразуване на 2-3-4 -дървета в еквивалентни на тях двоични дървета или R-D - дървета, алгоритмите за които са по-прости.



Дървета. В - дървета

В-дървета или дървета на Байер и МакКрейт (Bayer&McCreight)

Това са дървета с **много разклонения**. В-дърво може да се разглежда като обобщение на 2-3 и 2-3-4-дървета.

В-дърво от ред ***m*** се нарича дърво за претърсване със следните свойства:

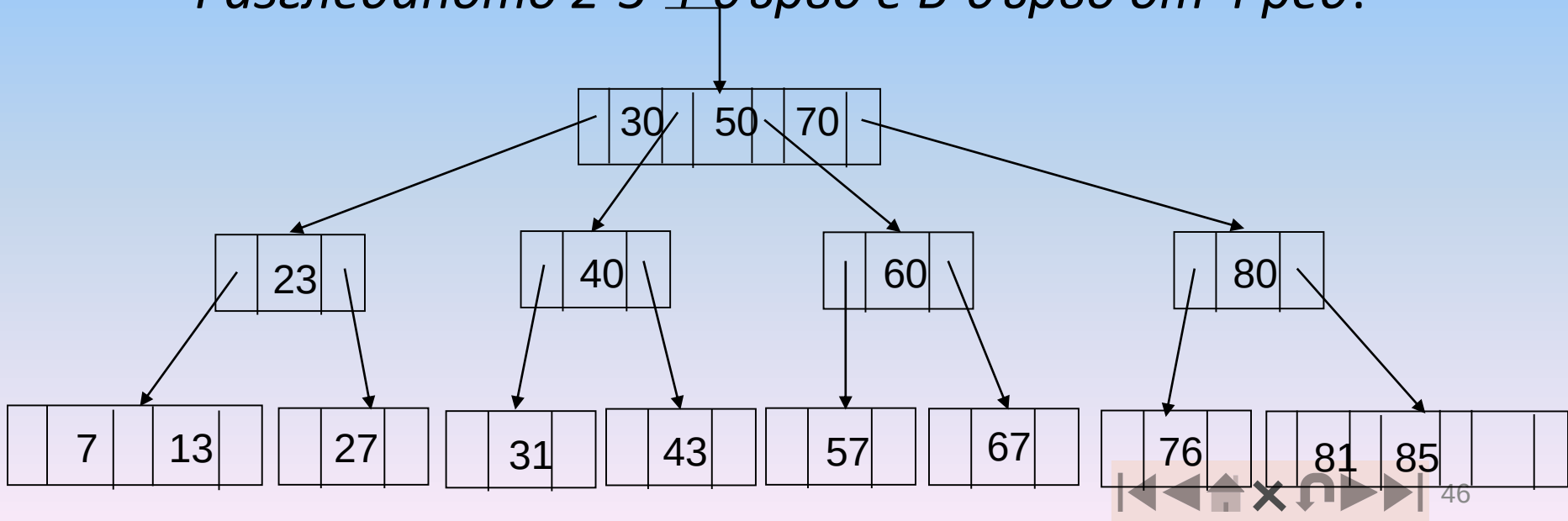
- За всички възли, освен корена, брой наследници ***i*** е
$$m/2 \leq i \leq m$$
- За корена брой наследници ***i*** е
$$2 \leq i \leq m;$$
- Всички пътища от корена до произволно листо имат еднаква дължина.



Дървета. В - дървета

Ако възел съдържа **n** полета с данни, той има **$i=n+1$** наследника. Максималния брой наследници е **m** . Всеки вътрешен възел, който не е корен, има поне **$m/2$** връзки (наследника), ако **m** е четно, и поне **$(m+1)/2$** връзки (наследника, ако **m** е нечетно, където **m** - е порядъка на дървото (или максималният брой връзки за всеки възел).

Разгледаното 2-3-4-дърво е В-дърво от 4 ред:





Дървета. Червено - черни дървета

Червено-черно дърво (Red – Black Trees) се нарича балансирано (но не идеално балансирано!) двоично дърво за претърсване (BST), в което всеки връх е означен или като червен, или като черен и допълнително са изпълнени следните **свойства**:

1. Всеки връх или е червен, или е черен;
2. Коренът е черен;
3. Всички външни възли (върхове със стойност NULL) са черни;
4. Ако даден връх е червен, то двата му наследника са черни, освен това всеки червен възел има черен родител;
5. Всички пътища от произволен връх T до произволно листо от поддървото с корен T съдържат еднакъв брой черни върхове.

